Completed

Non-Preemptive Shortest Job First CPU Scheduling Algorithm

Write a C program to implement the Non-Preemptive Shortest Job First (SJF) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), and Burst Times (BT). At any given time, the process with the minimum burst time among the arrived processes must be executed first. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and find the overall average TAT and WT.

Core Logic

Selection: Out of all processes that have arrived by the current time, the one with the minimum Burst Time is chosen.

Formulae:

$Turnaround Time (TAT) = Completion Time (CT) - Arrival Time (AT)$
$Waiting Time (WT) = Turnaround Time (TAT) - Burst Time (BT)$

nput Format

The first line of input contains an integer representing the number

Select Language : C (GCC 9.2.0)

```c
#include <stdio.h>

int main() {
    int n, i, completed = 0, time = 0, min_index;

    printf("Enter number of processes: \n");
    scanf("%d", &n);

    int AT[n], BT[n], CT[n], TAT[n], WT[n], done[n];

    for(i = 0; i < n; i++) {
        printf("Enter Arrival Time and Burst Time for P%d: \n", i + 1);
        scanf("%d %d", &AT[i], &BT[i]);
        done[i] = 0;
    }

    while(completed < n) {
        min_index = -1;
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

Completed

Non-Preemptive Shortest Job First CPU Scheduling Algorithm

Write a C program to implement the Non-Preemptive Shortest Job First (SJF) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), and Burst Times (BT). At any given time, the process with the minimum burst time among the arrived processes must be executed first. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and find the overall average TAT and WT.

ore Logic

election: Out of all processes that have arrived by the current me, the one with the minimum Burst Time is chosen.

rmulae:

urnaround Time (TAT) = Completion Time (CT) - Arrival Time (AT)$
aiting Time (WT) = Turnaround Time (TAT) - Burst Time (BT)$

ut Format

first line of input contains an integer representing the number

Select Language : C (GCC 9.2.0)

```c
        for(i = 0; i < n; i++) {
            if(!done[i] && AT[i] <= time) {
                if(min_index == -1 || BT[i] < BT[min_index])
                    min_index = i;
            }
        }


        if(min_index == -1) {
            time++;
        } else {
            time += BT[min_index];
            CT[min_index] = time;
            done[min_index] = 1;
            completed++;
        }
    }

```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

Back To Practice

Q1

Completed

Non-Preemptive Shortest Job First CPU Scheduling Algorithm

Write a C program to implement the Non-Preemptive Shortest Job First (SJF) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), and Burst Times (BT). At any given time, the process with the minimum burst time among the arrived processes must be executed first. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and find the overall average TAT and WT.

Core Logic

Selection: Out of all processes that have arrived by the current time, the one with the minimum Burst Time is chosen.

Formulae:

$Turnaround Time (TAT) = Completion Time (CT) - Arrival Time (AT)$
$Waiting Time (WT) = Turnaround Time (TAT) - Burst Time (BT)$

Input Format

The first line of input contains an integer representing the number

```
    }

    float totalTAT = 0, totalWT = 0;

    for(i = 0; i < n; i++) {
        TAT[i] = CT[i] - AT[i];
        WT[i] = TAT[i] - BT[i];
        totalTAT += TAT[i];
        totalWT += WT[i];
    }

    printf("\nAverage Turnaround Time = %.2f\n", totalTAT / n);
    printf("Average Waiting Time = %.2f\n", totalWT / n);

    return 0;
}
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

> Test Case - 2